Importing Libraries

In [1]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
```

TASK 1

In [2]:
```
def sample_image(image, sampling_factor):
    return image[::sampling_factor, ::sampling_factor]
```

In [3]:
```
def quantize_image(image, levels):
    quantized = np.floor(image / 256 * levels) * (256 / levels)
    return quantized.astype(np.uint8)
```

In [4]:
```
def plot_images(original, sampled, quantized):
    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(original, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(sampled, cmap='gray')
    plt.title('Sampled Image')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(quantized, cmap='gray')
    plt.title('Quantized Image')
    plt.axis('off')

    plt.show()
```

In [5]:
```
image_path = "images/lena_gray_256.tif"

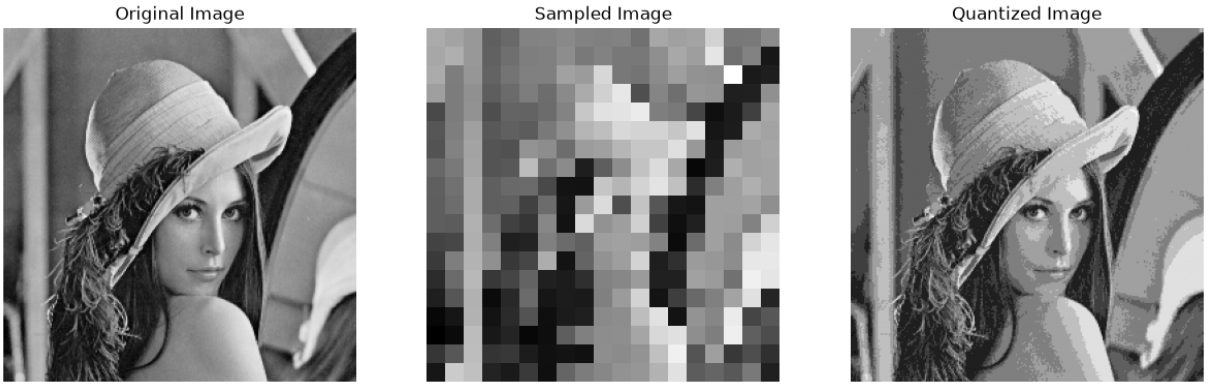
sampling_factor = 14
quantization_levels = 9

original_image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if original_image is None:
    print("Error: Unable to load image")

sampled_image = sample_image(original_image, sampling_factor)
quantized_image = quantize_image(original_image, quantization_levels)

plot_images(original_image, sampled_image, quantized_image)
```



In [6]:
```
sampling_factor = 4
quantization_levels = 4

sampled_image = sample_image(original_image, sampling_factor)
quantized_image = quantize_image(original_image, quantization_levels)

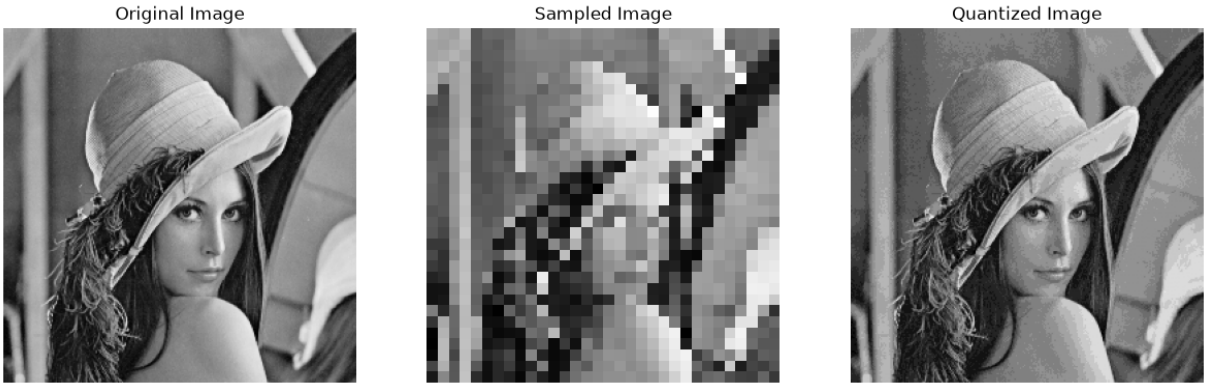
plot_images(original_image, sampled_image, quantized_image)
```



In [8]:
```
sampling_factor = 8
quantization_levels = 16

sampled_image = sample_image(original_image, sampling_factor)
quantized_image = quantize_image(original_image, quantization_levels)

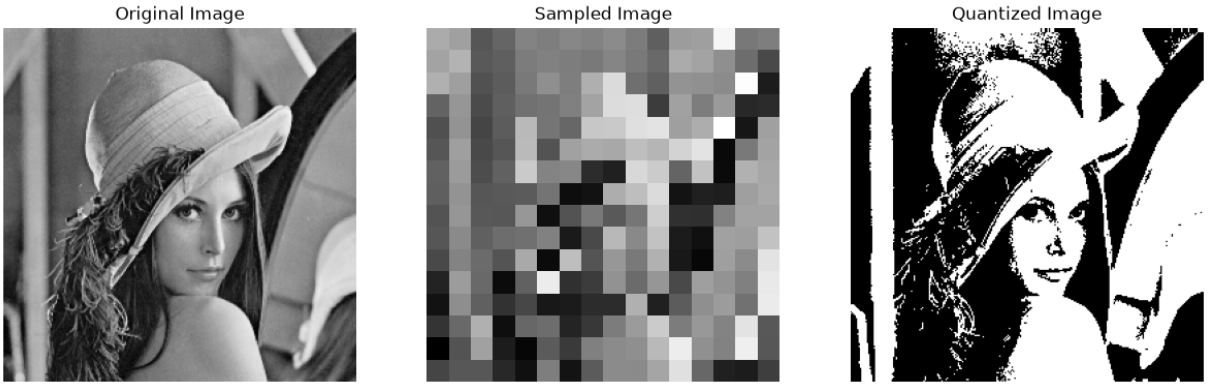
plot_images(original_image, sampled_image, quantized_image)
```



In [9]:
```
sampling_factor = 16
quantization_levels = 2

sampled_image = sample_image(original_image, sampling_factor)
quantized_image = quantize_image(original_image, quantization_levels)

plot_images(original_image, sampled_image, quantized_image)
```



TASK 2

In [10]:
```
img1 = Image.open("images/lena_gray_256.tif").convert("L")
img2 = Image.open("images/cameraman.tif").convert("L")

resize = (400, 400)

img1 = img1.resize(resize, Image.Resampling.LANCZOS)
img2 = img2.resize(resize, Image.Resampling.LANCZOS)

im1arr = np.asarray(img1)
im2arr = np.asarray(img2)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(im1arr, cmap='gray')
plt.title("Lena")
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
plt.imshow(im2arr, cmap='gray')
plt.title("Cameraman")
plt.axis("off")

plt.show()
```



In [11]:
```
subtraction = cv2.subtract(im1arr, im2arr)

plt.figure(figsize=(5, 5))
plt.imshow(subtraction, cmap='gray')
plt.title("Subtraction")
plt.axis("off")
plt.show()
```



In [12]:
```
addition_175 = cv2.add(im1arr, 175)

plt.figure(figsize=(5, 5))
plt.imshow(addition_175, cmap='gray')
plt.title("Image + 175")
plt.axis("off")
plt.show()
```



In [14]:
```
A = cv2.imread("images/A.jpg", cv2.IMREAD_GRAYSCALE)
B = cv2.imread("images/B.jpg", cv2.IMREAD_GRAYSCALE)

if A is None or B is None:
    print("Error: Unable to load A or B")

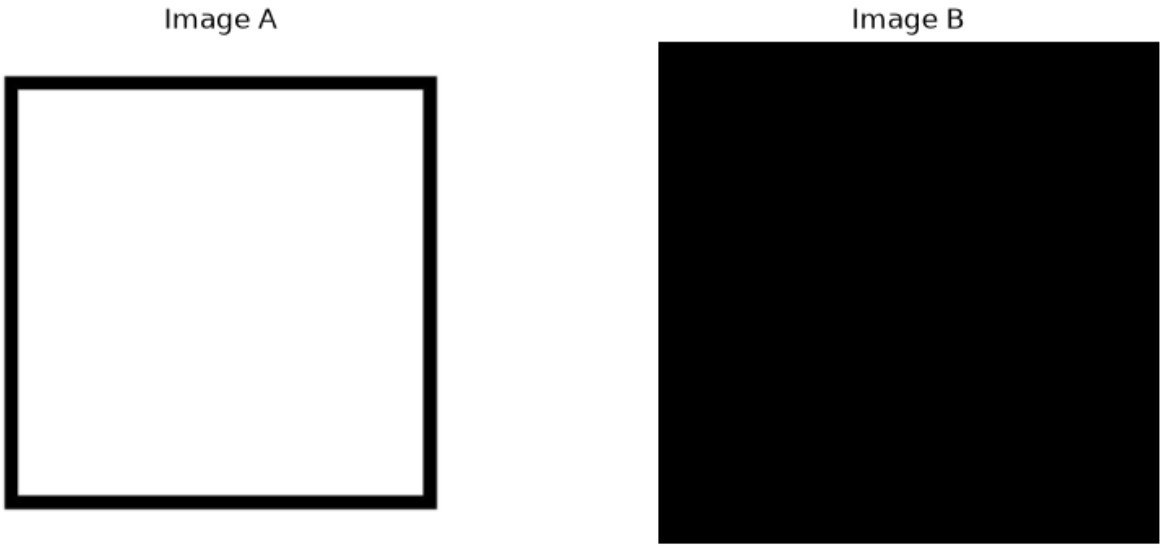
A = cv2.resize(A, (400, 400))
B = cv2.resize(B, (400, 400))

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(A, cmap='gray')
plt.title("Image A")
plt.axis("off")

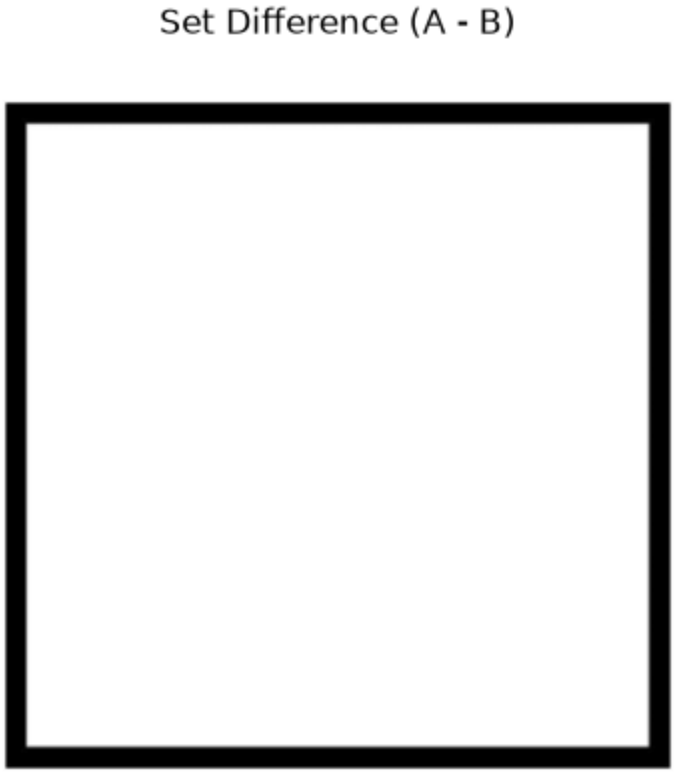
plt.subplot(1, 2, 2)
plt.imshow(B, cmap='gray')
plt.title("Image B")
plt.axis("off")

plt.show()
```



In [15]:
```
set_difference = cv2.bitwise_and(A, cv2.bitwise_not(B))

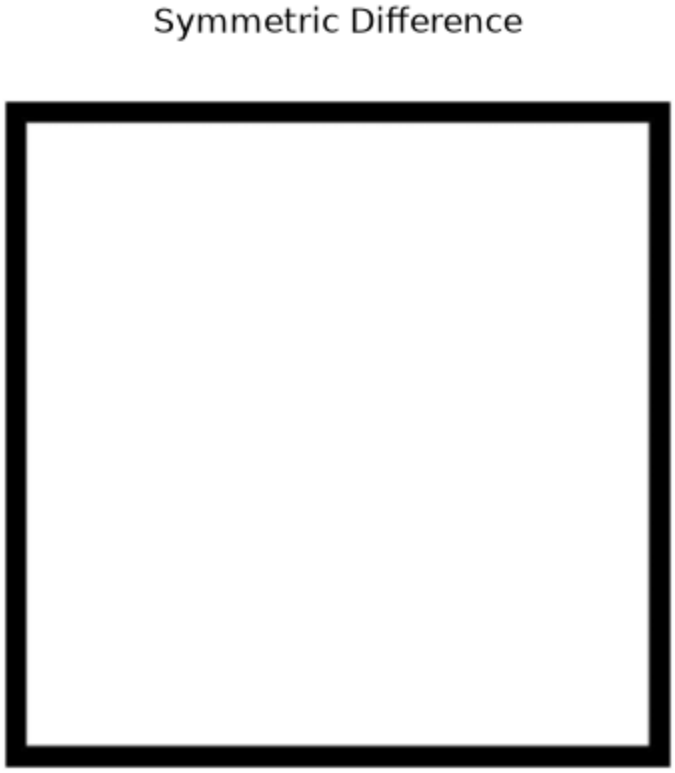
plt.figure(figsize=(5, 5))
plt.imshow(set_difference, cmap='gray')
plt.title("Set Difference (A - B)")
plt.axis("off")
plt.show()
```



In [16]:
```
symmetric_difference = cv2.bitwise_xor(A, B)

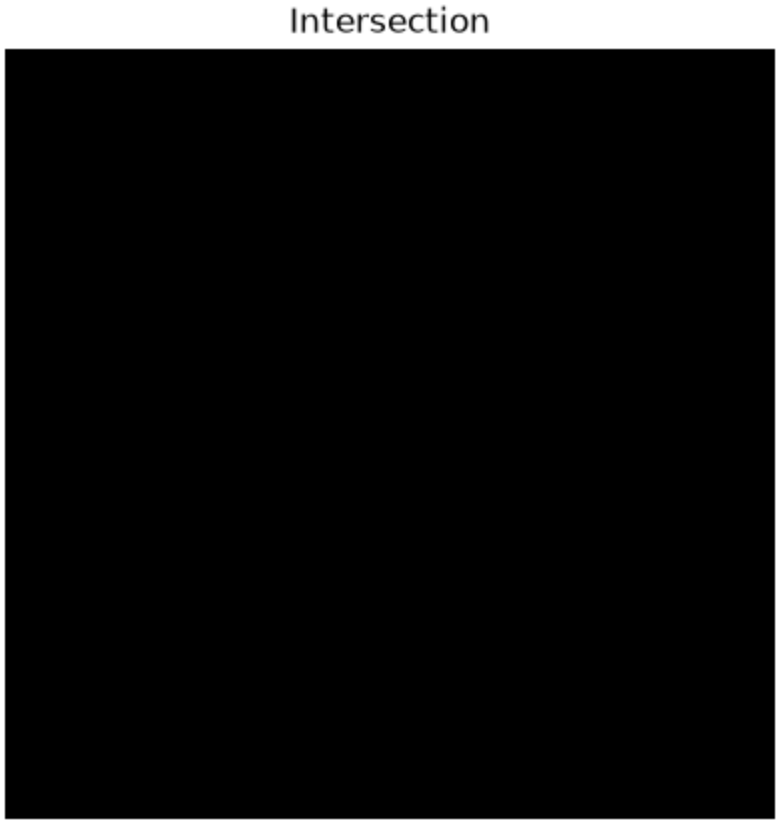
plt.figure(figsize=(5, 5))
plt.imshow(symmetric_difference, cmap='gray')
```

```
plt.title("Symmetric Difference")
plt.axis("off")
plt.show()
```



In [17]:
```
intersection = cv2.bitwise_and(A, B)

plt.figure(figsize=(5, 5))
plt.imshow(intersection, cmap='gray')
plt.title("Intersection")
plt.axis("off")
plt.show()
```



In [18]:
```
union = cv2.bitwise_or(A, B)

plt.figure(figsize=(5, 5))
plt.imshow(union, cmap='gray')
plt.title("Union")
plt.axis("off")
plt.show()
```

Union

